

PROPOSAL TUGAS AKHIR

1. Judul : Analisis Performa dan Pemanfaatan *Bloom Filter* untuk *Whitelisting* pada Arsitektur *Microservices*
2. Deskripsi : Arsitektur *microservices* merupakan pendekatan dalam pengembangan perangkat lunak yang membagi aplikasi menjadi layanan-layanan kecil dan independen, di mana setiap layanan menangani fungsi bisnis tertentu, dapat dikembangkan serta diimplementasikan secara terpisah, dan berkomunikasi melalui protokol ringan. Pendekatan ini mendukung skalabilitas, fleksibilitas, serta pengelolaan data yang terdesentralisasi sehingga meningkatkan efektivitas dan efisiensi sistem [1]. Namun, lonjakan volume data yang masif telah memberikan tantangan baru terhadap kinerja *microservices*, khususnya dalam hal kemampuan untuk tetap responsif dan stabil di bawah tekanan beban tinggi serta arsitektur data yang kompleks [2]. Salah satu aspek krusial dalam konteks ini adalah proses validasi permintaan, yang umumnya dilakukan melalui mekanisme *whitelisting* untuk memastikan hanya entitas yang dikenal dan valid yang diizinkan mengakses layanan sistem, sementara semua entitas lain secara default ditolak [3].

Whitelisting umumnya diimplementasikan melalui kueri langsung ke database sebagai sumber kebenaran utama (*source of truth*). Akan tetapi, validasi langsung ke database memaksa setiap permintaan yang masuk untuk bersaing memperebutkan koneksi dan siklus pemrosesan yang terbatas, sehingga semakin meningkatkan latensi dan berisiko menimbulkan perlambatan berantai pada layanan-layanan yang bergantung padanya [4]. Ketergantungan penuh pada akses database secara langsung dalam lingkungan dengan tingkat konkurensi tinggi pada akhirnya dapat menimbulkan kerentanan serius, baik berupa penurunan performa maupun meningkatnya risiko keamanan sistem [5]. Akumulasi permintaan validasi yang terus-menerus menghantam database menyebabkan antrian yang panjang dan lonjakan drastis pada latensi respons p95 maupun p99 saat terjadi lonjakan trafik, sehingga berdampak negatif terhadap throughput layanan [6]. Akibatnya, database tidak hanya menjadi titik pusat kemacetan tetapi juga berpotensi menjadi titik kegagalan tunggal yang menghambat skalabilitas seluruh sistem.

Untuk mengatasi permasalahan tersebut, salah satu pendekatan yang banyak diteliti adalah pemanfaatan struktur data probabilistik, yakni struktur data yang memiliki efisiensi ruang dan kemampuan memberikan hasil aproksimasi secara cepat [7]. Salah satu contoh paling dikenal adalah *Bloom Filter*, yang digunakan untuk pengujian keanggotaan himpunan secara efisien dengan memanfaatkan kombinasi fungsi hash. Meskipun *Bloom*

Filter sangat hemat ruang dan mampu memberikan hasil dengan cepat, teknik ini memiliki keterbatasan berupa kemungkinan terjadinya false positive, yakni kondisi ketika suatu elemen dinyatakan ada padahal sebenarnya tidak [8].

Sifat ini menjadikan *Bloom Filter* kandidat ideal untuk berfungsi sebagai lapisan penyaringan awal, dan *Bloom Filter* maupun bit vector filter lainnya telah digunakan secara luas dalam sistem manajemen basis data untuk melakukan reduksi data sejak tahap awal. Pendekatan ini menyediakan cara yang efisien untuk secara probabilistik mengeliminasi baris data lebih awal, sehingga mengurangi jumlah baris yang perlu diproses lebih lanjut sekaligus meningkatkan kinerja kueri [9]. *Bloom Filter* dapat dimanfaatkan secara efektif dalam metode *whitelisting* untuk menyaring entitas yang tidak valid sebelum mencapai database.

Penelitian ini akan direalisasikan melalui pengembangan proyek mandiri berupa layanan *whitelist* berbasis Golang dan gRPC, yang berdiri sendiri dan dapat dipanggil langsung oleh *microservice* lain untuk melakukan validasi *whitelisting* secara real-time. Validasi dilakukan sepenuhnya melalui layanan ini dengan dua skenario berbeda. Pada skenario pertama, setiap permintaan diteruskan langsung ke PostgreSQL sebagai database sumber kebenaran. Pada skenario kedua, layanan akan menggunakan *Bloom Filter* sebagai lapisan penyaring awal sebelum meneruskan ke PostgreSQL. Pemilihan mode validasi ini dapat dikendalikan melalui konfigurasi yang dapat diubah secara fleksibel tanpa perlu memodifikasi kode. Lingkungan pengujian dijalankan pada kontainer Docker dengan monitoring berbasis Prometheus dan Grafana untuk mengukur performa.

3. Tools/Tech Stack :
 - 1) Golang, bahasa pemrograman utama untuk membangun layanan.
 - 2) gRPC, pustaka untuk komunikasi antar-*microservices*.
 - 3) RabbitMQ, message broker untuk komunikasi asinkron antar-*microservices*.
 - 4) Bloom Filter, lapisan penyaring awal untuk *whitelisting*.
 - 5) PostgreSQL, database utama sebagai sumber kebenaran (*source of truth*).
 - 6) k6, alat uji beban untuk mengukur performa gRPC.
 - 7) Grafana Stack, untuk logging, monitoring, dan visualisasi metrik.
 - 8) Docker & Kubernetes, kontainerisasi dan orkestrasi layanan.
 - 9) Google Cloud Platform, infrastruktur cloud untuk pengujian skala besar.
4. Mitra/Perusahaan : Laboratorium TRPL
5. Daftar Pustaka : [1] Oyeniran, O., Adewusi, A., Adeleke, A., Akwawa, L., & Azubuko, C. (2024). Microservices architecture in cloud-native

applications: Design patterns and scalability. *Computer Science & IT Research Journal*. <https://doi.org/10.51594/csitrj.v5i9.1554>.

[2] Guntupalli, B., & Surya Vamshi, C. (2023). Designing microservices that handle high-volume data loads. *International Journal of AI, Big Data, Computational and Management Studies*, 4(4), 76–87. <https://doi.org/10.63282/3050-9416.IJAIBDCMS-V4I4P109>.

[3] Sedgewick, A., Souppaya, M., & Scarfone, K. (2015). Guide to Application Whitelisting. <https://doi.org/10.6028/NIST.SP.800-167>.

[4] Li, T., Butrovich, M., Ngom, A., Lim, W., McKinney, W., & Pavlo, A. (2020). Mainlining Databases: Supporting Fast Transactional Workloads on Universal Columnar Data File Formats. *ArXiv*, abs/2004.14471. <https://doi.org/10.14778/3436905.3436913>.

[5] Zhang, Y. (2025). Research on Optimization and Security Management of Database Access Technology in the Era of Big Data. *Academic Journal of Computing & Information Science*. <https://doi.org/10.25236/ajcis.2025.080102>.

[6] Deshpande, R. A. (2025). Approaches to reducing REST API response time in high-traffic banking systems. *International Journal of Scientific Engineering and Science*, 9(8), 63–68. <https://ijses.com/wp-content/uploads/2025/08/68-IJSES-V9N8.pdf>

[7] Chang, A. (2020, October 23). Taking advantage of probabilistic data structures. *Aerospike*. <https://aerospike.com/blog/taking-advantage-of-probabilistic-data-structures/>

[8] Alsuhbany, S. A., Alsuhaibani, M., Khan, R. U., & Qamar, A. M. (2022). Performance Analysis of Bloom Filter for Big Data Analytics. *Computational intelligence and neuroscience*, 2022, 2414605. <https://doi.org/10.1155/2022/2414605>

[9] Zeyl, T., Cheng, Q., Pournaghi, R., Lam, J., Wang, W., Wong, C., Chen, C., & Larson, P. (2025). Including Bloom Filters in Bottom-up Optimization. <https://doi.org/10.1145/3722212.3724440>.